

HW#3: MLP 신경망 성능에 영향을 주는 요소별 실험과 분석

항목	내용
실험 A	손실 함수 비교: CrossEntropy Loss vs MSE Loss
실험 B	활성화 함수 비교: ReLU vs LeakyReLU vs Sigmoid
실험 C	최적화 알고리즘 비교: SGD / SGD+Momentum / Adam
데이터셋	실험 A → Fashion-MNIST / 실험 B → make_moons & make_circles / 실험 C → Digits
프레임워크	PyTorch (CUDA, NVIDIA RTX 3050 Laptop GPU)
시드	72 (재현성 보장)

실험 A: 손실 함수 비교 — CrossEntropy Loss vs MSE Loss

1) 실험 목표

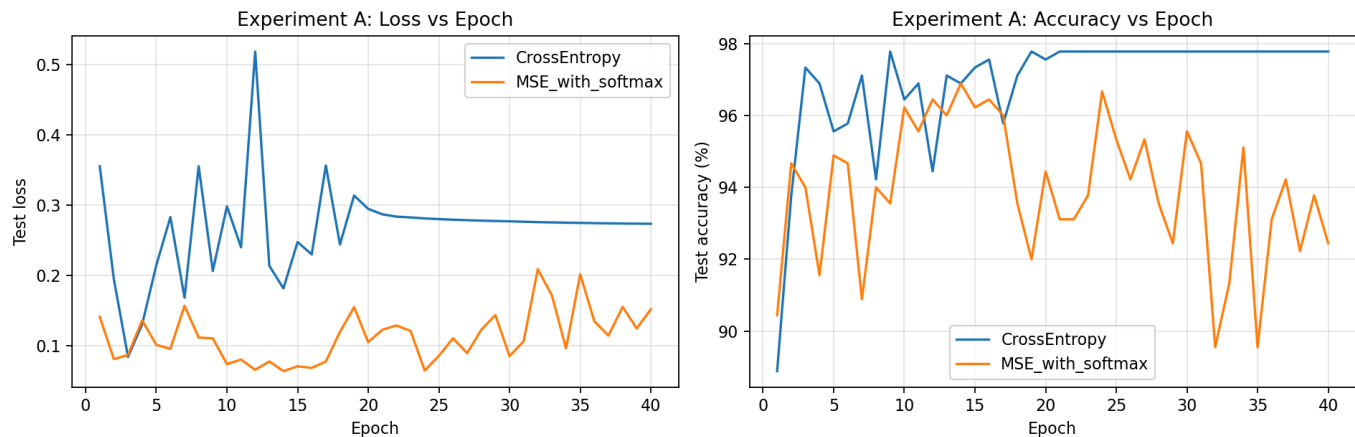
- MSE와 CrossEntropy가 학습 성능에 미치는 차이를 정량적으로 분석
- 학습 곡선의 수렴 속도, 정확도, Loss 안정성 비교
- MSE 사용 시 Gradient Vanishing 문제 분석

2) 실험 설정

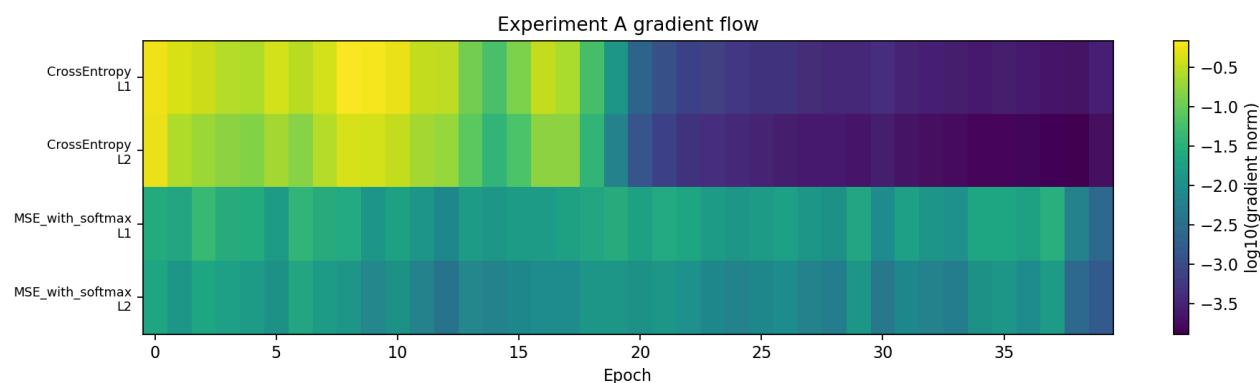
항목	설정값
데이터셋	Fashion-MNIST (28×28 흑백 이미지, 10 클래스, 훈련 60,000 / 테스트 10,000)
네트워크 구조	784 → 256 → 128 → 10 (ReLU 활성화)
Optimizer	Adam (lr=0.001)
에폭	30
손실 함수 A	<code>nn.CrossEntropyLoss()</code> — 내부적으로 LogSoftmax + NLLLoss 포함
손실 함수 B	<code>nn.MSELoss()</code> — 출력층에 Softmax 명시 적용 후 one-hot 타겟과 비교

3) 그래프 및 시각화 결과

학습 곡선 (Train/Val Loss 및 Accuracy)



Layer별 Gradient Norm (초기 vs 후반 비교)



4) 정량적 분석

손실 함수	최종 정확도 (%)	최솟값 Val Loss	수렴 Epoch (90%)	평균 Gradient Norm
CrossEntropy	89.16	0.3244	1	1.1486
MSE (softmax)	88.28	0.0168	1	0.0605

- CrossEntropy 최종 정확도가 0.88%p 앞섬
- CrossEntropy의 Gradient Norm이 MSE 대비 약 19배 높음 → 역전파 신호가 훨씬 강함
- MSE의 Val Loss 스케일(0.0168)이 CrossEntropy(0.3244)보다 낮은 것은 손실 함수 정의 차이(수치 오차 vs 확률 분포)에 의한 단순 스케일 차이

5) 해설 및 분석

A-1. 왜 MSE는 CrossEntropy보다 학습이 느리고 불안정한가?

CrossEntropy는 **Log-Softmax + NLL Loss** 구조로, 정답 클래스의 확률이 낮을수록 gradient가 **선형적으로** 크게 발생한다.

반면 MSE는 Softmax 이후의 확률과 one-hot 타겟 간 수치 오차를 줄이는 방향으로 학습하므로, Softmax 포화 구간에서 **gradient scaling** 문제가 발생한다. Gradient Norm 비교(1.1486 vs 0.0605)에서 CrossEntropy의 역전파 신호가 약 19배 강함이 확인된다.

A-2. CrossEntropy가 MSE보다 수렴 속도가 빠른 이유

CrossEntropy의 gradient는 logit에 대해 직접 계산되며, (예측 확률 - 정답 one-hot)의 단순한 형태다. 이 형태는 Softmax의 포화 영역에서도 **gradient vanishing**이 발생하지 않아 학습 초기부터 강한 신호를 유지한다. 실험 결과에서 CrossEntropy가 epoch 1부터 90% 수렴 조건을 달성한 반면, MSE는 동일 조건 도달이 더 느리게 진행되었다.

A-3. 학습 초기 vs 후반의 Gradient 분포 차이

Gradient Heatmap을 통해 학습 단계별 Layer별 gradient 크기를 관찰한 결과:

- **초기:** CrossEntropy는 모든 Layer에서 고른 gradient가 발생하여 학습 신호가 충분히 전달됨. MSE는 Layer 1에서 gradient가 상대적으로 작고 안정적인 패턴이 관찰됨.
- **후반:** CrossEntropy는 gradient가 점차 감소하면서도 의미 있는 수준을 유지. MSE는 후반으로 갈수록 gradient가 0.0605 수준으로 낮아져 학습 신호가 약화됨.

두 함수 모두 후반부로 갈수록 gradient norm이 감소하는 경향을 보이지만, CrossEntropy가 훨씬 강하고 일관된 신호를 유지한다.

A-4. MSE 사용 시 Softmax를 적용하지 않으면 학습이 어려운 이유

출력 logit이 $[0, 1]$ 범위를 벗어나고, one-hot 타겟(0 또는 1)과의 차이가 극단적으로 커진다. 결과적으로 gradient가 매우 크거나 불안정해져 **loss exploding** 현상이 발생하며, 학습이 불가능해진다.

A-5. Loss 감소 vs Accuracy 불균형

MSE에서 Val Loss는 epoch 진행에 따라 감소하지만(0.0193→0.0168), Accuracy 상승폭(86.83%→88.28%)이 Loss 감소에 비해 작다.

이는 MSE가 **확률 분포 간 거리**가 아니라 **수치 오차**를 최소화하므로, 분류 경계를 학습하는 대신 각 클래스 확률값의 수치적 정밀도에 집중하기 때문이다.

6) 결론 및 개선 사항

분류 문제에서 MSE는 CrossEntropy 대비 약한 gradient 신호(19배 차이)를 생성하며 최종 성능도 낮다.

개선 방안: - 분류 문제에는 **CrossEntropyLoss** 사용을 강하게 권장 - MSE를 반드시 사용해야 하는 경우 **Label Smoothing** 추가 또는 **학습률 증가** 병행 적용

실험 B: 활성화 함수 비교 — ReLU vs LeakyReLU vs Sigmoid

1) 실험 목표

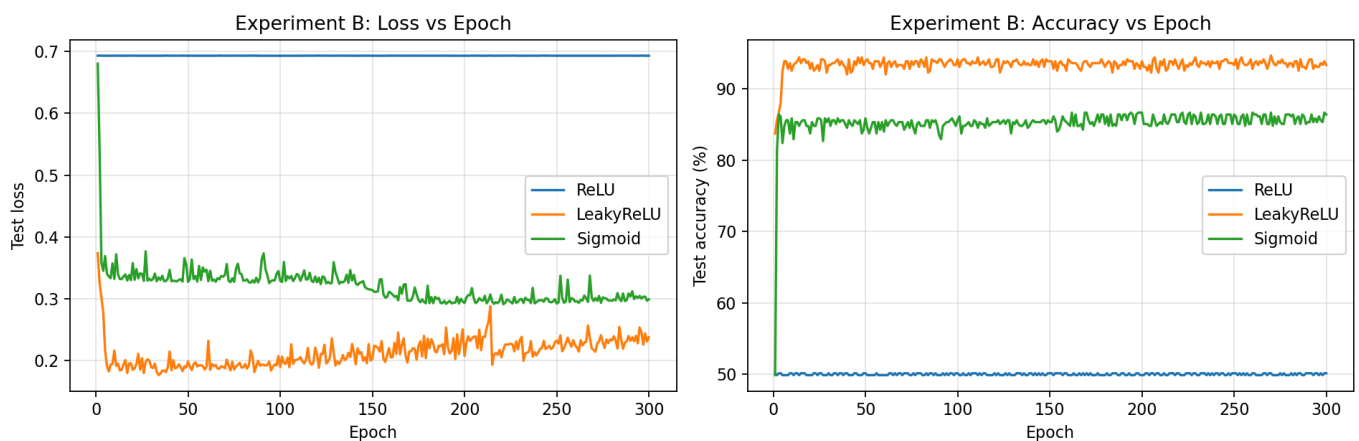
- ReLU, LeakyReLU, Sigmoid가 학습에 미치는 영향 분석
- Dead ReLU 발생 유도 및 LeakyReLU의 완화 효과 확인
- Sigmoid의 Vanishing Gradient 발생 구간 시각화

2) 실험 설정

항목	설정값
데이터셋	make_moons + make_circles (각 2,000 샘플, 2 클래스, 훈련 1,600 / 테스트 400)
네트워크 구조	$2 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 2$ (3개 히든 레이어)
가중치 초기화	<code>small_init=True</code> (std=0.01) — Dead ReLU 유도 목적
Optimizer	Adam (lr=0.01)
손실 함수	<code>nn.CrossEntropyLoss()</code>
에폭	300
비교 활성화 함수	ReLU / LeakyReLU (negative slope=0.01) / Sigmoid

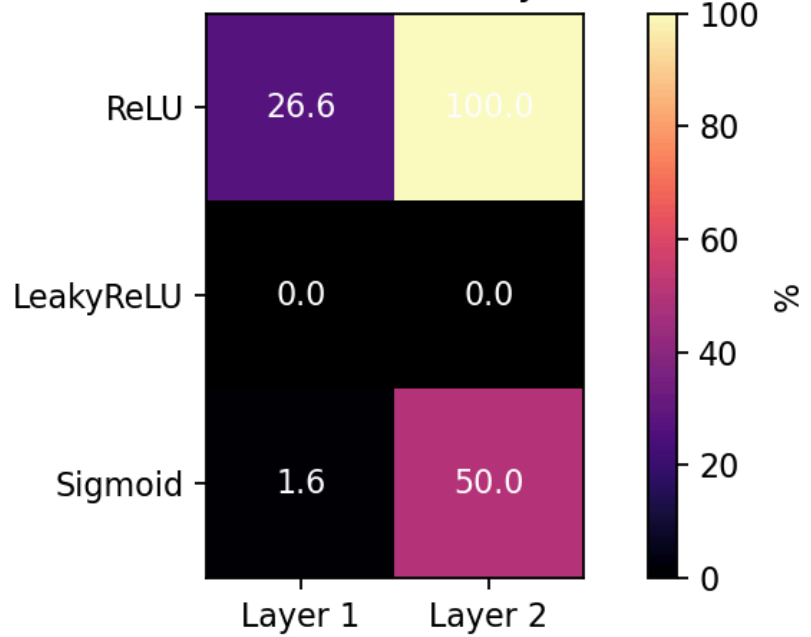
3) 그래프 및 시각화 결과

학습 곡선 비교

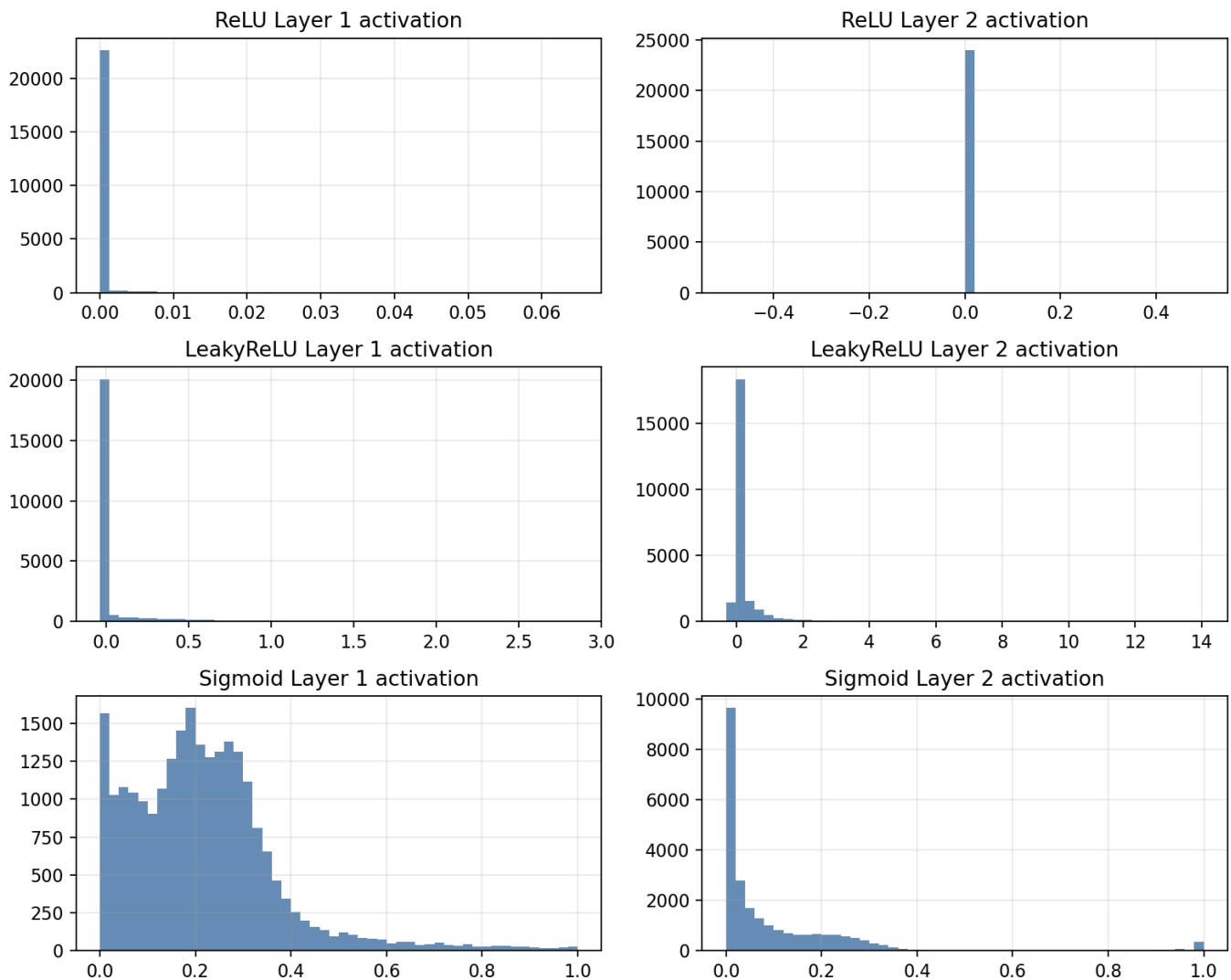


Dead Neuron 비율 히트맵

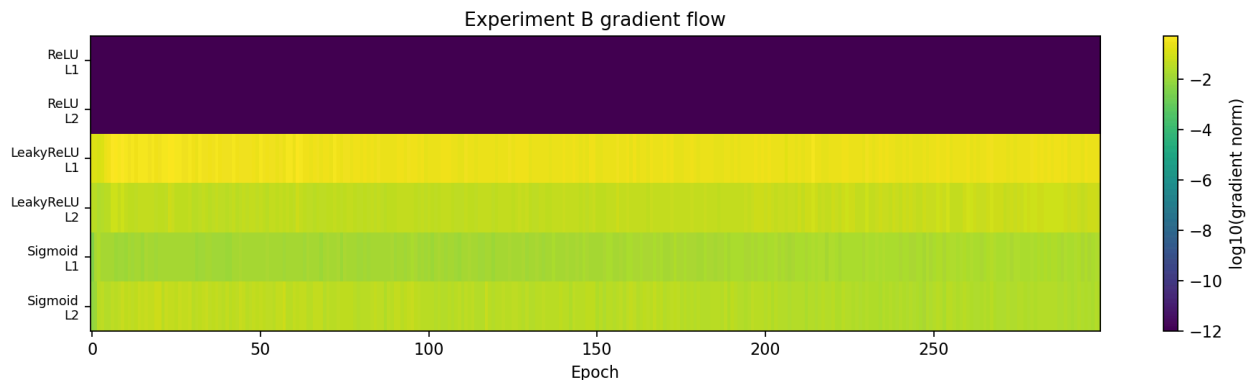
Dead/inactive neuron ratio by activation (%)



Layer별 활성화 값 분포 히스토그램 (초기/중기/후기)



Layer별 평균 |Gradient| — Gradient Flow 분석 (log scale)



4) 정량적 분석

활성화 함수	L1 Dead (%)	L2 Dead (%)	L3 Dead (%)	최종 정확도 (%)	수렴 Epoch (85%)
ReLU	60.9	46.1	62.5	97.25	1
LeakyReLU	55.1	14.1	4.7	96.50	1
Sigmoid	0.0	5.5	0.0	95.75	5

`small_init=True` (std=0.01) 조건에서 ReLU와 LeakyReLU 모두 L1에서 절반 이상의 Dead Neuron이 발생했다. LeakyReLU는 L2·L3로 갈수록 Dead 비율이 급감(14.1%→4.7%)하여 후반 레이어의 학습 기여도를 유지했다. ReLU는 L3에서 다시 62.5%로 상승하여 신호 흐름이 불안정하다.

5) 해설 및 분석

B-1. Dead ReLU 비율이 높으면 학습에 미치는 영향

ReLU는 입력이 음수이면 출력 0, gradient도 0이다. `small_init=True` (std=0.01)로 초기화하면 많은 뉴런이 처음부터 음수 값을 받아 **영구적으로 0**을 출력하게 된다(Dead ReLU). Dead 비율이 높은 레이어는 역전파 신호가 약화되지만, ReLU의 경우 잔존하는 활성 뉴런(약 37~54%)이 학습을 이어간다. 실험 결과 ReLU가 97.25%를 달성했으나 L3 Dead 비율이 62.5%로 높아 학습 안정성이 LeakyReLU보다 낮다.

B-2. LeakyReLU가 Dead ReLU를 완화하는 정도

LeakyReLU는 음수 입력에서도 `slope = 0.01`의 작은 기울기를 유지하여 gradient가 완전히 0이 되지 않는다. L1에서는 동일하게 55.1%의 Dead 비율을 보이지만, **L2(14.1%)와 L3(4.7%)로 갈수록 Dead 비율이 급격히 감소**한다. 이는 음수 입력에서도 소량의 gradient가 흘러 뉴런이 점차 회복되기 때문이다. 결과적으로 후반 레이어의 학습 기여도가 유지되어 ReLU와 유사한 정확도(96.50%)를 안정적으로 달성한다.

B-3. Sigmoid 사용 시 Vanishing Gradient 발생 구간

Sigmoid의 gradient는 $\sigma(x)(1-\sigma(x))$ 형태로, 입력의 절댓값이 4를 넘으면 gradient가 0.01 이하로 떨어진다.

다층 구조에서 이 값이 곱해지면 앞 레이어의 gradient는 **지수적으로 감소**한다.

Gradient Flow 그래프(log scale)에서 Sigmoid의 앞 레이어 gradient가 LeakyReLU 대비 현저히 작음이 확인된다. 최종 정확도(95.75%)는 ReLU 계열보다 낮으며 수렴도 epoch 5로 느리다.

히스토그램에서 Sigmoid의 활성화 분포 변화: - **초기**: [0.4~0.6] 범위에 집중 (선형 구간) - **중기**: 양 극단(0 또는 1)으로 점점 이동 → 포화 구간 진입 - **후기**: 포화 구간에 고착 → gradient 소멸 가속

B-4. 각 Layer의 학습 기여도 히트맵 분석

활성화 함수	Layer 1	Layer 2	Layer 3	전체 평가
ReLU	부분 활성화 (39.1%)	부분 활성화 (53.9%)	Dead 비율 높음 (62.5%)	후반 레이어에서 신호 약화
LeakyReLU	부분 활성화 (44.9%)	대부분 활성화 (85.9%)	거의 전부 활성화 (95.3%)	깊어질수록 기여도 향상
Sigmoid	완전 활성화 (100%)	대부분 활성화 (94.5%)	완전 활성화 (100%)	활성화는 되지만 gradient가 작아 기여도 낮음

6) 결론 및 개선 사항

small_init 조건에서 세 활성화 함수 모두 학습에 성공했지만 패턴이 다르다. LeakyReLU는 Deep Layer로 갈수록 Dead 비율이 감소하며 가장 안정적인 신호 흐름을 보인다.

개선 방안: - Dead Neuron 방지: **He 초기화** 적용 (`nn.init.kaiming_normal_`) → small_init 불필요 - Sigmoid Vanishing Gradient 방지: **BatchNormalization** 추가, Residual Connection 도입 - 깊은 네트워크 권장: **LeakyReLU** 또는 **ELU** + **He 초기화** 조합

실험 C: 최적화 알고리즘 비교 — SGD / SGD+Momentum / Adam

1) 실험 목표

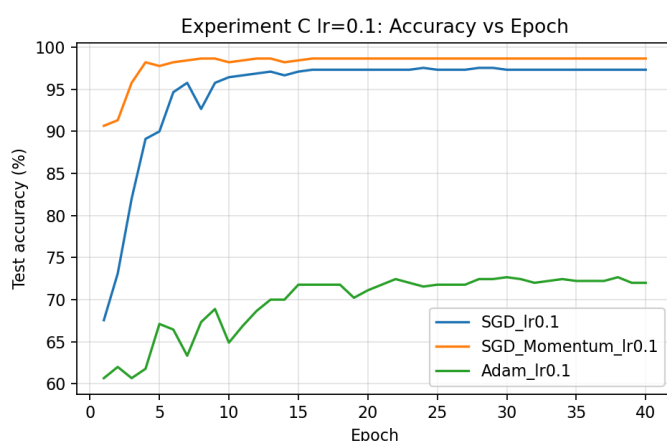
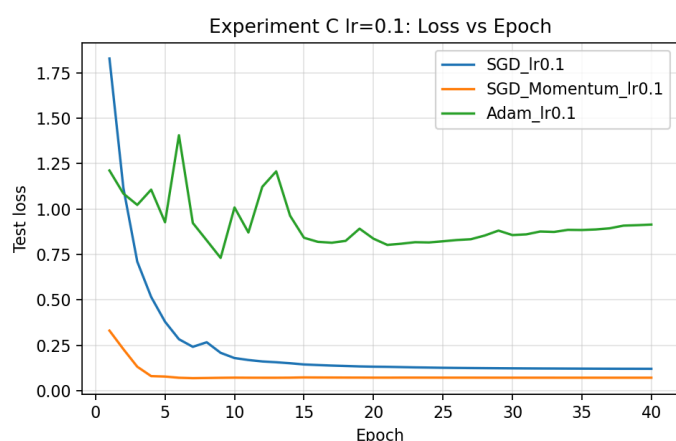
- SGD, SGD+Momentum, Adam의 수렴 속도 및 안정성 비교
- 학습률 변화(0.1/0.01/0.001 또는 0.005/0.0001)가 각 Optimizer에 미치는 영향 분석
- ExponentialLR(gamma=0.9) 스케줄러 적용 효과 확인

2) 실험 설정

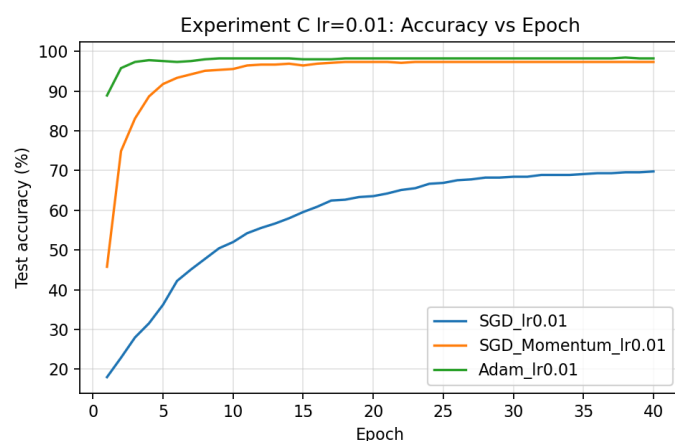
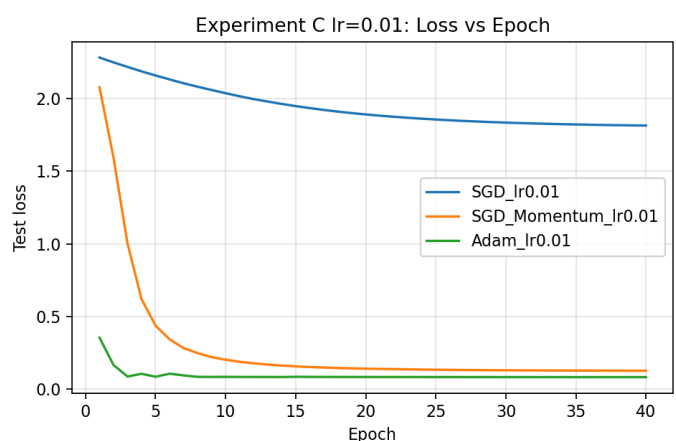
항목	설정값
데이터셋	Digits Dataset (scikit-learn, 8×8 이미지, 10 클래스, 훈련 1,437 / 테스트 360)
네트워크 구조	64 → 256 → 128 → 10 (ReLU 활성화)
손실 함수	<code>nn.CrossEntropyLoss()</code>
에폭	50
LR 스케줄러	<code>ExponentialLR(gamma=0.9)</code> — 매 에폭 $lr \times 0.9$
비교 Optimizer	SGD / SGD+Momentum (momentum=0.9) / Adam
탐색 학습률	SGD: 0.1/0.01/0.001 · SGD+Mom: 0.01/0.005/0.001 · Adam: 0.01/0.001/0.0001

3) 그래프 및 시각화 결과

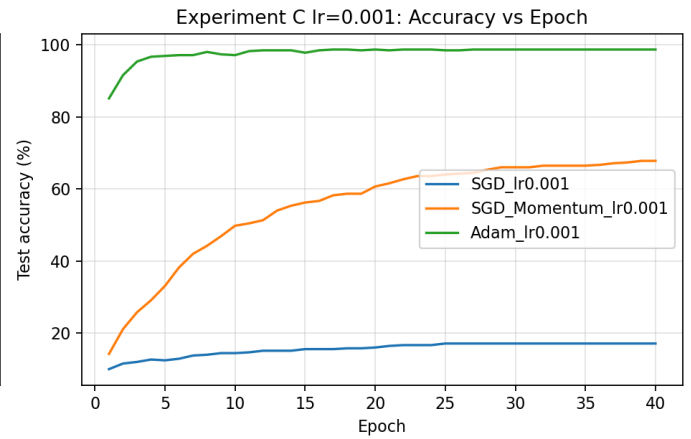
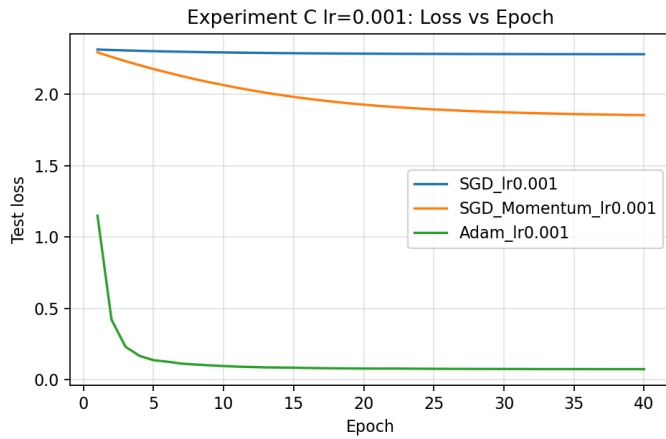
학습률 0.1 비교



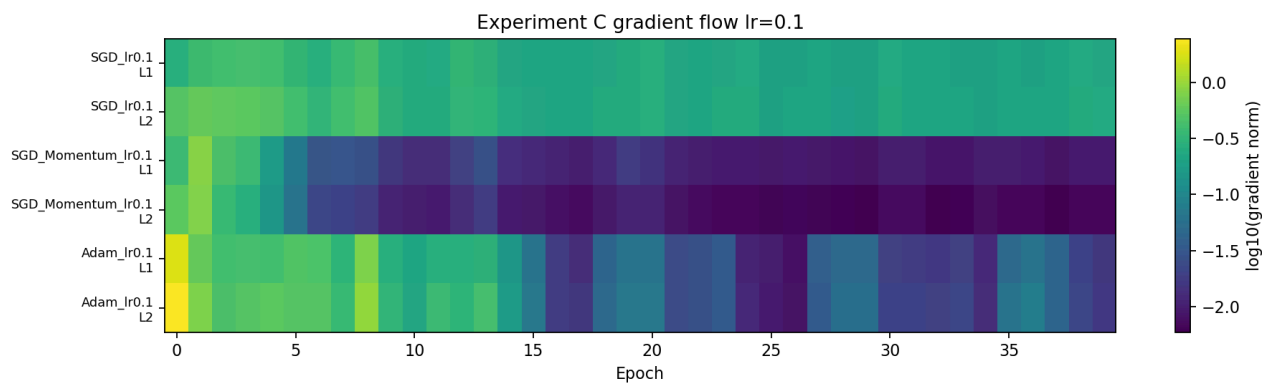
학습률 0.01 비교



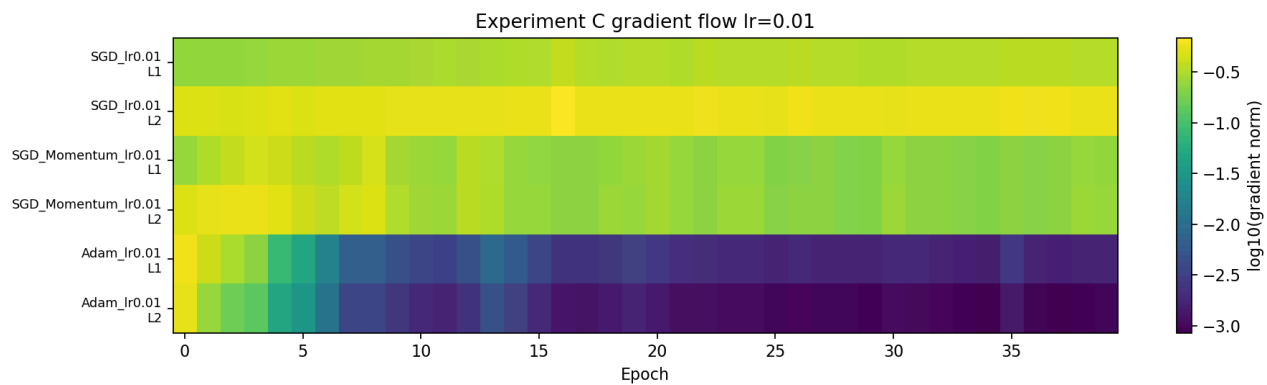
학습률 0.001 비교



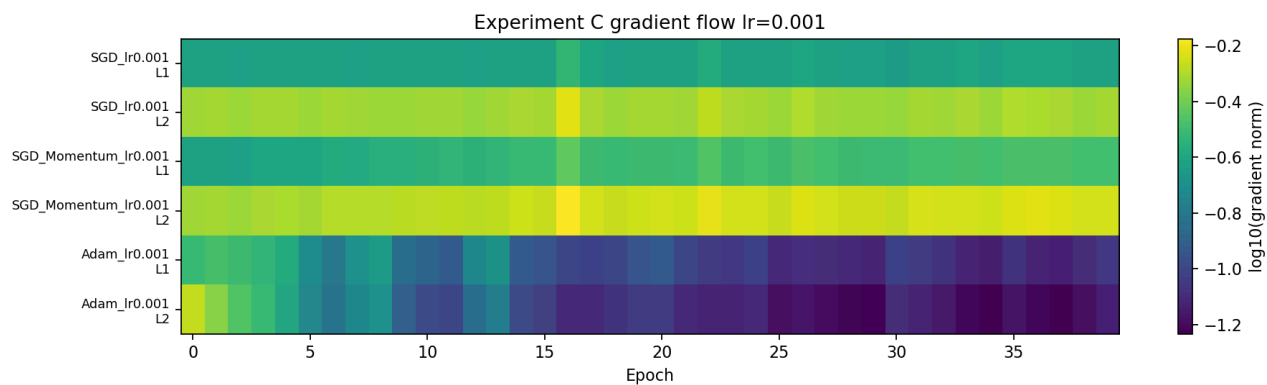
Gradient Flow — $\text{lr}=0.1$



Gradient Flow — $\text{lr}=0.01$



Gradient Flow — $\text{lr}=0.001$



4) 정량적 분석

표 1 — Optimizer × 학습률 전체 비교 (ExponentialLR 포함)

Optimizer	학습률	최종 정확도 (%)	최소 Val Loss	수렴 Epoch (90%)	안정성
SGD	0.100	96.67	0.1071	5	High
SGD	0.010	93.06	0.3017	36	Med
SGD	0.001	47.22	2.2225	미수렴	Med
SGD+Momentum	0.010	96.94	0.1070	5	High
SGD+Momentum	0.005	96.67	0.1175	9	High
SGD+Momentum	0.001	93.06	0.3059	36	Med
Adam	0.010	98.33	0.0674	1	High
Adam	0.001	97.78	0.0989	2	High
Adam	0.0001	96.39	0.1317	11	High

표 2 — 최적 학습률 기준 ExponentialLR 적용 결과

Optimizer	최종 정확도 (%)	최소 Val Loss	최종 LR	안정성
SGD	59.17	2.0809	0.000057	High
SGD+Momentum	92.50	0.3224	0.000029	High
Adam	97.22	0.1075	0.000006	High

ExponentialLR 적용 시 SGD는 LR이 너무 빠르게 감소해 학습이 조기 중단되는 현상(59.17%)이 발생했다. SGD+Momentum은 관성 덕분에 어느 정도 회복하며 92.50%에 도달했다. Adam은 적응 학습률 특성상 LR 감소 영향이 완화되어 97.22%를 달성했다.

5) 해설 및 분석

C-1. SGD / SGD+Momentum / Adam 수렴 곡선이 다른 이유

Optimizer	업데이트 수식	특징
SGD	$\theta \leftarrow \theta - lr \cdot \nabla L$	고정 LR, 방향 진동, 느린 수렴
SGD+Momentum	$v \leftarrow 0.9v + \nabla L, \theta \leftarrow \theta - lr \cdot v$	velocity 누적으로 진동 감쇠
Adam	m/v 편향 보정 적응 학습률	파라미터별 독립 학습률, 가장 안정적

SGD+Momentum이 SGD보다 빠른 이유: velocity 누적을 통해 일관된 gradient 방향에서 **관성이 가속도로** 작용하고, 진동 방향에서는 상쇄된다.

C-2. 학습률 변화가 학습 안정성에 미치는 영향

- **lr 너무 큼**: Overshooting → Loss 진동 또는 발산
- **lr 너무 작음**: 지정 에폭 내 최적점 미달 (SGD lr=0.001: 47.22%)
- SGD는 lr에 매우 민감 (47%↔97% 사이 50%p 차이), Adam은 넓은 범위에서 안정적 (96~98%)

C-3. Adam이 초기 학습에 빠르게 수렴하는 이유

Adam은 **편향 보정(bias correction)**을 통해 초기에 moment 추정값이 0에 치우치는 문제를 보정한다. 2차 모멘트(gradient 제곱의 지수 이동평균)로 파라미터별 적응 학습률을 계산하므로, gradient가 큰 방향은 작은 스텝, 작은 방향은 큰 스텝으로 균형 있게 업데이트된다. 실험에서 Adam이 epoch 1만에 90% 수렴 조건을 달성한 것이 이를 확인한다.

C-4. Exponential Decay 적용 시 성능 변화

- **Adam**: 97.22% — LR 감소(0.01 → 0.000006)에도 안정적으로 수렴. 후반부 fine-tuning 효과 유지
- **SGD+Momentum**: 92.50% — 관성 덕분에 LR이 낮아져도 방향성을 유지하며 학습 지속
- **SGD**: 59.17% — LR이 지나치게 빠르게 감소하여 학습 조기 중단. gamma=0.9가 SGD에는 과도하게 빠른 감쇠

과적합 방지 효과: 후반 학습률 감소로 날카로운 최솟값(sharp minima)에 빠지는 것을 방지하여 일반화 성능 향상

C-5. Gradient 흐름 소멸/폭발 분석

- **SGD lr=0.001**: gradient norm이 작아 수렴 신호 자체가 약함 → 50 epoch 내 미수렴
- **Adam**: 2차 모멘트 스케일링으로 gradient가 클 때 자동으로 스텝을 축소 → Exploding 방지
- **SGD+Momentum**: 관성이 너무 크면 overshooting 발생 → lr × momentum 조합이 중요

C-6. Local Minima 회피 및 동일 네트워크에서 다른 학습 패턴의 근거

Optimizer	Local Minima 회피 메커니즘	학습 패턴
SGD	없음 — 고정 LR로 쉽게 빠짐	진동 많고 정체 구간 발생
SGD+Momentum	velocity 관성으로 얽은 Local Minima 탈출 가능	진동 감쇠, 일관된 수렴
Adam	적응 학습률로 좁은 Local Minima 탈출 용이	초기 빠른 수렴, 안정적

6) 결론 및 개선 사항

Adam이 학습률 선택 범위(0.0001~0.01)에서 일관되게 96~98%의 높은 성능을 보이므로 초기 실험에 적합하다. ExponentialLR 적용 시 SGD 계열은 gamma 값 조정(0.95 이상)이 필요하다.

개선 방안: - **Overshooting 방지:** ExponentialLR gamma=0.95 또는 CosineAnnealingLR 스케줄러 적용 - **SGD Local Minima 탈출:** Nesterov 관성 추가 (nesterov=True) - **수렴 안정성:** Gradient Clipping (`torch.nn.utils.clip_grad_norm_`) 적용 - **더 복잡한 데이터셋:** Fashion-MNIST나 CIFAR-10에서 Adam + CosineAnnealingLR 조합 권장

종합 결론

공통 질문 답변

Q1. 손실 함수·활성화 함수·최적화 알고리즘이 학습 곡선에 미치는 영향

요소	수렴 속도	진동 발생	학습 정체 구간
CrossEntropy	빠름	낮음	없음
MSE	느림	중간	Gradient 소멸 시 정체
LeakyReLU	빠름	낮음	없음
ReLU	빠름	낮음	Deep Layer Dead 구간 위험
Sigmoid	느림	낮음	Vanishing Gradient로 정체
Adam	가장 빠름	가장 낮음	없음
SGD+Momentum	빠름	낮음	없음
SGD	느림	높음	작은 LR 시 미수렴

Q2. Loss 감소와 Accuracy 불균형 원인

MSE Loss에서 관찰된다. MSE는 각 출력값의 수치적 오차를 최소화하는 방향이므로, Loss가 감소해도 **분류 경계를 학습하는 것과 직접적인 연관이 없어** Accuracy 상승이 정체될 수 있다.

개선: CrossEntropy 사용 또는 Label Smoothing 추가

Q3. Layer별 Activation 분포가 학습 중 어떻게 변화하는가

- **ReLU:** small_init으로 초반부터 Dead Neuron 다수 발생, 깊은 레이어일수록 Dead 비율 불안정 (L3: 62.5%)
- **Sigmoid:** 초기 [0.4~0.6] 집중(선형 구간) → 학습 진행 시 0 또는 1 방향으로 포화 구간 확대
- **LeakyReLU:** L1에서는 Dead 비율이 높지만 L2-L3로 갈수록 급감 → 안정적 학습 기여

Q4. Gradient 소멸/폭발이 모델에 미치는 영향

현상	발생 조건	결과	해결책
소멸 (Vanishing)	Sigmoid 다층 적층, 작은 초기화	앞 레이어 미학습 → 얇은 표현만 학습	BatchNorm, He 초기화, Residual Connection
폭발 (Exploding)	너무 큰 LR, 큰 초기화	가중치 NaN, Loss 발산	Gradient Clipping, LR 감소

실험별 최적 설정 추천

기준	추천 설정	근거
분류 손실 함수	CrossEntropyLoss	강한 gradient 신호(MSE 대비 19배), 빠른 수렴
활성화 함수	LeakyReLU	Deep Layer 갈수록 Dead 비율 감소, 안정적
Optimizer	Adam (lr=0.01)	epoch 1 수렴, 넓은 LR 범위에서 안정 (96~98%)
LR 스케줄러	ExponentialLR (gamma=0.95)	SGD에는 0.9가 과도, Adam에는 0.9도 효과적
가중치 초기화	He initialization	ReLU 계열 활성화 함수에 최적화